

REAL TIME WEATHER DATA RETRIEVAL, VISUALIZATION & ANALYSIS USING PYTHON

Hmm... I wonder what the weather is like today? 🤔



WEATHER ALERT

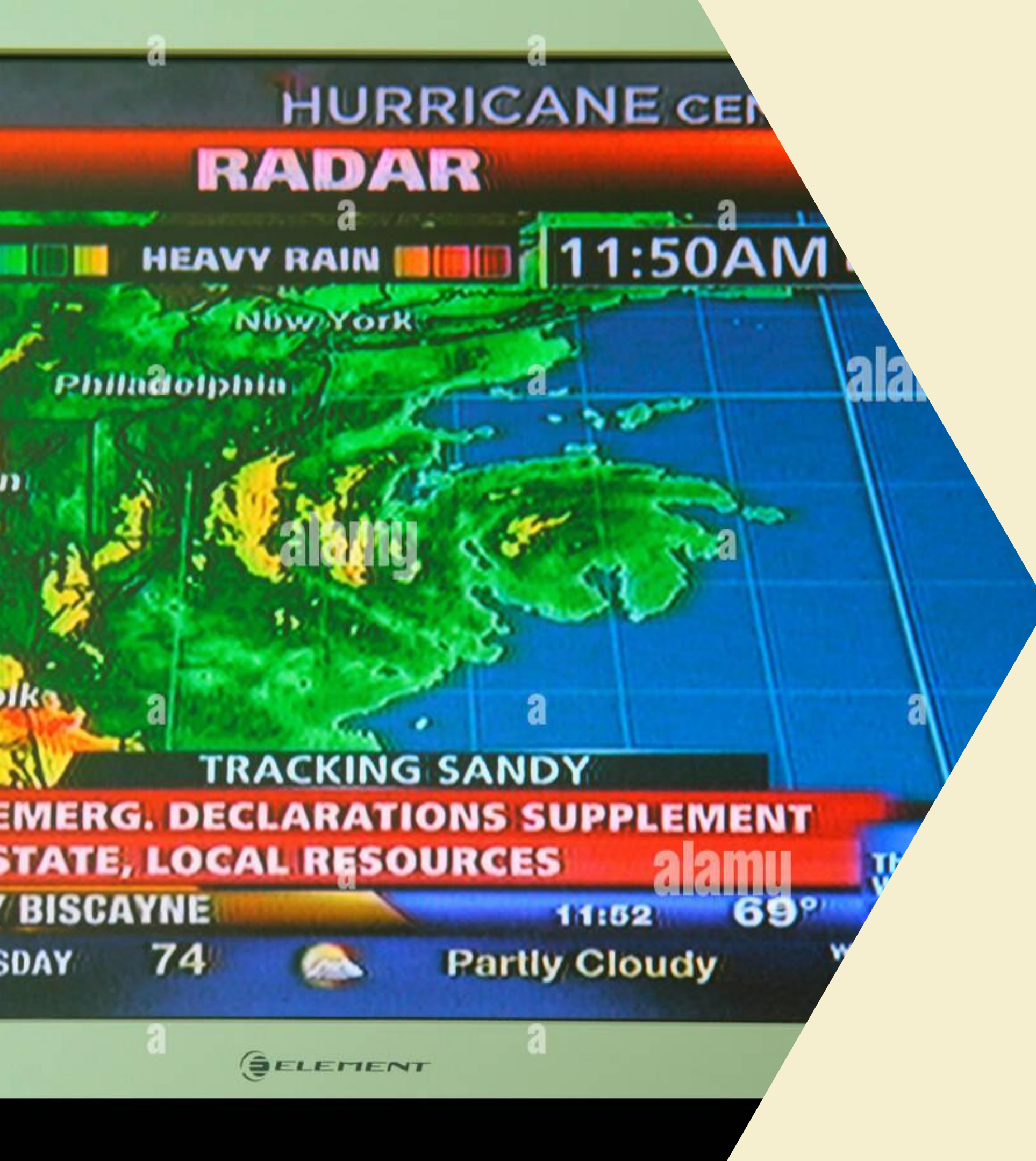
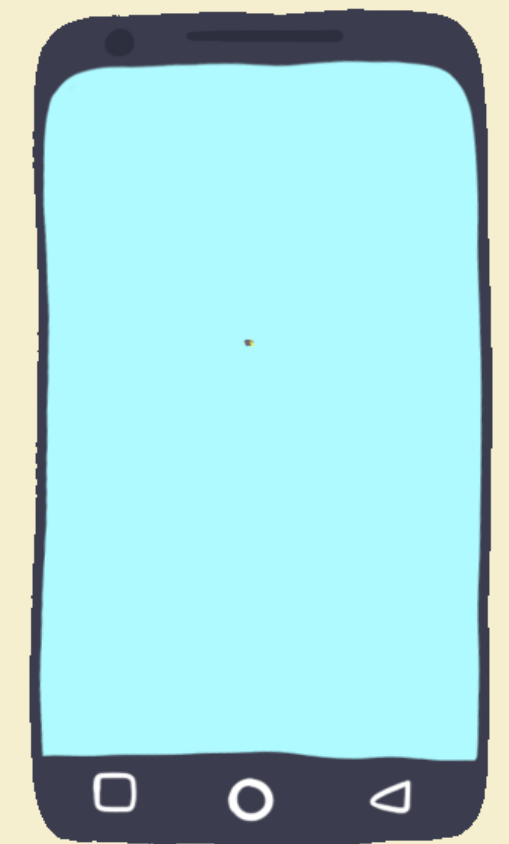


Table Of Contents

- 1.Introduction
- 2.Objectives
- 3.Modules
- 4.Source Code
- 5.Challenges Faced
- 6.Conclusion



Introduction

Our Weather Monitoring and Visualization System provides a comprehensive solution to track and analyze real-time weather data from around the globe. This system leverages the OpenWeather API to collect key meteorological information, such as temperature, humidity, wind speed, UV index, and more, for multiple cities worldwide.

Additionally, the system utilizes several powerful libraries to visualize and enhance the data:

- **Folium**: Used to create interactive weather maps, providing a visual representation of weather conditions in various cities. It allows users to explore weather data on a geographical map.
- **Plotly and Matplotlib**: These libraries are used for data visualization, helping users compare and analyze temperature and humidity trends across different cities with bar and line charts.
- **Seaborn**: A Python data visualization library based on Matplotlib, used to create aesthetically pleasing and informative plots of weather trends.
- **Schedule**: This library automates the process of fetching and updating weather data at regular intervals, ensuring the system stays up-to-date without manual intervention.
- **Requests**: This library is used to interact with the OpenWeather API, fetching weather data and handling HTTP requests.

Key Features

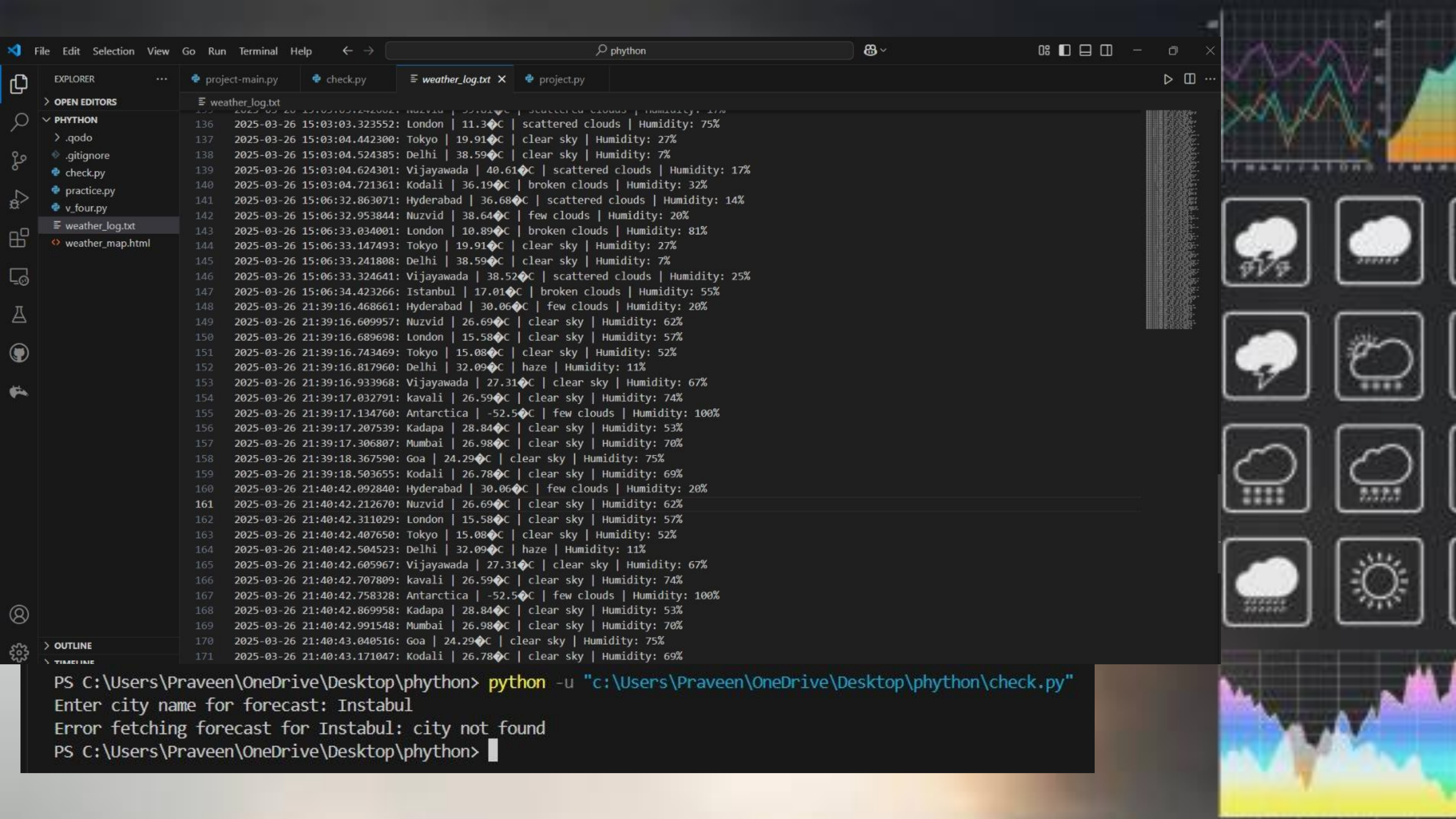
- **Real-time Weather Updates:** The system regularly fetches current weather data, including temperature, wind speed, and weather conditions, for a variety of cities. It updates the weather information every 2 minutes, providing timely insights.
- **Weather Forecasting:** In addition to real-time data, the system also retrieves weather forecasts for specific cities.
- **Interactive Weather Maps:** Using Folium, the system creates interactive maps that visualize weather data for multiple cities.
- **Temperature and Humidity Comparison:** The application compares temperature and humidity levels across different cities using Plotly and Matplotlib.





5. **Wind Chill and Sunlight Duration:** The system calculates additional weather parameters, such as wind chill (for colder regions with higher wind speeds) and sunlight duration (the time between sunrise and sunset), providing more insights into daily weather conditions.

6. **Data Logging:** Every weather update is logged in a text file, which allows users to maintain a historical record of weather conditions.




```

check.py > ...
1  import requests
2  import schedule
3  import time
4  import folium
5  import plotly.graph_objects as go
6  import matplotlib.pyplot as plt
7  import seaborn as sns
8  from datetime import datetime, timezone, timedelta
9
10 API_KEY = "22ba1a51558faee7656eb9a1c55f176a"
11

```

Qodo Gen: Options | Test this function

```

def sunlight_duration(sunrise_str, sunset_str):
    sunrise = datetime.strptime(sunrise_str, "%Y-%m-%d %H:%M:%S")
    sunset = datetime.strptime(sunset_str, "%Y-%m-%d %H:%M:%S")
    duration = sunset - sunrise
    return str(duration)

```

Qodo Gen: Options | Test this function

```

def temp_status(temp):
    if temp < 10:
        return "❄️ Cold"
    elif 10 <= temp <= 25:
        return "😊 Pleasant"
    else:
        return "🔥 Hot"

```

```

PS C:\Users\Praveen\OneDrive\Desktop\python> python -u "c:\Users\Praveen\OneDrive\Desktop\python\check.py"
Enter city name for forecast: Kodali
Weather updated at: 2025-03-26 21:40:42
Hyderabad: 30.06°C, few clouds ☁️ Wind Chill: N/A°C, 🔥 Hot, Daylight: 12:12:27
Nuzvid: 26.69°C, clear sky 🌤️ Wind Chill: N/A°C, 🔥 Hot, Daylight: 12:12:13
London: 15.58°C, clear sky 🌤️ Wind Chill: N/A°C, 😊 Pleasant, Daylight: 12:33:37
Tokyo: 15.08°C, clear sky 🌤️ Wind Chill: N/A°C, 😊 Pleasant, Daylight: 12:22:39
Delhi: 32.09°C, haze 🌫️ Wind Chill: N/A°C, 🔥 Hot, Daylight: 12:17:10
Vijayawada: 27.31°C, clear sky 🌤️ Wind Chill: N/A°C, 🔥 Hot, Daylight: 12:12:07
kavali: 26.59°C, clear sky 🌤️ Wind Chill: N/A°C, 🔥 Hot, Daylight: 12:11:33
Antarctica: -52.5°C, few clouds ☁️ Wind Chill: -68.94°C, ❄️ Cold, Daylight: 11:06:05
Kadapa: 28.84°C, clear sky 🌤️ Wind Chill: N/A°C, 🔥 Hot, Daylight: 12:11:23
Mumbai: 26.98°C, clear sky 🌤️ Wind Chill: N/A°C, 🔥 Hot, Daylight: 12:13:05
Goa: 24.29°C, clear sky 🌤️ Wind Chill: N/A°C, 😊 Pleasant, Daylight: 12:11:42
Kodali: 26.78°C, clear sky 🌤️ Wind Chill: N/A°C, 🔥 Hot, Daylight: 12:12:00
Map saved as weather_map.html. Open it in a browser.

```

BREAKING NEWS

HURRICANE LUSI THIS WEEKEND


```
def get_weather(city):
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"
    response = requests.get(url)
    data = response.json()

    if response.status_code == 200:
        return {
            "city": city,
            "temperature": data["main"]["temp"],
            "feels_like": data["main"].get("feels_like"),
            "temp_min": data["main"].get("temp_min"),
            "temp_max": data["main"].get("temp_max"),
            "humidity": data["main"]["humidity"],
            "pressure": data["main"]["pressure"],
            "sea_level": data["main"].get("sea_level"),
            "grnd_level": data["main"].get("grnd_level"),
            "weather": data["weather"][0]["description"],
            "wind_speed": data["wind"].get("speed"),
            "wind_deg": data["wind"].get("deg"),
            "gust": data["wind"].get("gust"),
            "rain_1h": data.get("rain", {}).get("1h", 0),
            "clouds": data["clouds"].get("all"),
            "lat": data["coord"]["lat"],
            "lon": data["coord"]["lon"],
            "sunrise": convert_to_ist(data["sys"].get("sunrise")),
            "sunset": convert_to_ist(data["sys"].get("sunset")),
            "visibility": data.get("visibility"),
            "country": data["sys"].get("country"),
        }
    else:
        print(f"Error fetching weather for {city}: {data['message']}")
        return None
```

Qodo Gen: Options | Test this function

```
def live_weather_update(cities):
    print("Weather updated at:", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
    cities_data = []
    for city in cities:
        data = get_weather(city)
        if data:
            wind_speed_kmh = data['wind_speed'] * 3.6
            wind_chill = calculate_wind_chill(data['temperature'], wind_speed_kmh)
            status = temp_status(data['temperature'])
            duration = sunlight_duration(data['sunrise'], data['sunset'])
            print(f"{data['city']}: {data['temperature']}°C, {data['weather']} {get_weather_emoji(data['weather'])}, Wind Chill: {wind_chill}°C, {s}
            log_to_file(data)
            cities_data.append(data)

    create_map(cities_data)
    plot_weather_trends(cities_data)
    plot_humidity(cities_data)
```

```
if __name__ == "__main__":
    cities = ["Hyderabad", "Nuzvid", "London", "Tokyo", "Delhi", "Vijayawada", "kavali", "Antarctica", "Kadapa", "Mumbai", "Goa"]
    city_for_forecast = input("Enter city name for forecast: ")
    if city_for_forecast not in cities:
        cities.append(city_for_forecast)
    live_weather_update(cities)
    plot_forecast(city_for_forecast)
    schedule_weather_updates(cities)
```

Qodo Gen: Options | Test this function

```
def calculate_wind_chill(temp_c, wind_speed_kmh):
    if temp_c <= 10 and wind_speed_kmh > 4.8:
        wc = 13.12 + 0.6215 * temp_c - 11.37 * (wind_speed_kmh ** 0.16) + 0.3965 * temp_c * (wind_speed_kmh ** 0.16)
        return round(wc, 2)
    else:
        return "N/A"
```

BREAKING NEWS

24/07

HURRICANE LUSI THIS WEEKEND

Qodo Gen: Options | Test this function

```
def log_to_file(data):  
    with open("weather_log.txt", "a") as f:  
        f.write(f"{datetime.now()}: {data['city']} | {data['temperature']}°C | {data['weather']} | Humidity: {data['humidity']}%\n")
```

Qodo Gen: Options | Test this function

Qodo Gen: Options | Test this function

```
def convert_to_ist(utc_timestamp):  
    utc_time = datetime.fromtimestamp(utc_timestamp, timezone.utc)  
    return utc_time.astimezone(IST).strftime("%Y-%m-%d %H:%M:%S")
```

Qodo Gen: Options | Test this function

```
def create_map(cities_data):  
    avg_lat = sum([data["lat"] for data in cities_data if data]) / len(cities_data)  
    avg_lon = sum([data["lon"] for data in cities_data if data]) / len(cities_data)  
    weather_map = folium.Map(location=[avg_lat, avg_lon], zoom_start=3)  
  
    for data in cities_data:  
        if data:  
            emoji = get_weather_emoji(data['weather'])  
            uv_index = get_uv_index(data['lat'], data['lon'])  
            duration = sunlight_duration(data['sunrise'], data['sunset'])  
            popup_html = f"""  
<b>City:</b> {data['city']} ({data['country']})<br>  
  
<b>{emoji} Weather:</b> {data['weather'].capitalize()}, Clouds: {data['clouds']}%<br>  
  
<b>🌡️ Temperature:</b> {data['temperature']}°C (Feels like: {data['feels_like']}°C)<br>  
  
<b>▼ Min / ▲ Max Temp:</b> {data['temp_min']}°C / {data['temp_max']}°C<br>  
  
<b>💧 Humidity:</b> {data['humidity']}%<br>  
  
<b>🌀 Wind:</b> {data['wind_speed']} m/s at {data['wind_deg']}°<br>  
  
<b>🌧️ Rain (last 1h):</b> {data['rain_1h']} mm<br>  
  
<b>📶 Pressure:</b> {data['pressure']} hPa<br>  
  
<b>☀️ UV Index:</b> {uv_index}<br>  
  
<b>🕒 Daylight:</b> {duration}<br>  
  
<b>👁️ Visibility:</b> {data.get('visibility', 'N/A')} meters<br>  
  
<b>🌅 Sunrise:</b> {data['sunrise']} | <b>🌇 Sunset:</b> {data['sunset']}  
            """
```

BREAKING NEWS

24/07

HURRICANE LUSI THIS WEEKEND


```

<b>☀️ UV Index:</b> {uv_index}<br>

<b>🕒 Daylight:</b> {duration}<br>

<b>👁️ Visibility:</b> {data.get('visibility', 'N/A')} meters<br>

<b>🌅 Sunrise:</b> {data['sunrise']} | <b>🌇 Sunset:</b> {data['sunset']}
"""
folium.Marker(
    location=[data["lat"], data["lon"]],
    popup=folium.Popup(popup_html, max_width=300),
    tooltip=f"{emoji} {data['city']}"
).add_to(weather_map)

weather_map.save("weather_map.html")
print("Map saved as weather_map.html. Open it in a browser.")

```

Qodo Gen: Options | Test this function

```

def plot_weather_trends(cities_data):
    fig = go.Figure()

    for data in cities_data:
        if data:
            fig.add_trace(go.Bar(
                x=[data["city"]],
                y=[data["temperature"]],
                name=f"{data['city']} ({data['weather']})"
            ))

    fig.update_layout(title="Temperature Comparison", xaxis_title="City", yaxis_title="Temperature (°C)")
    fig.show()

```

BREAKING NEWS

24/07 HURRICANE LUSI THIS WEEKEND

Qodo Gen: Options | Test this function

```
def get_weather_emoji(description):  
    description = description.lower()  
    if "cloud" in description:  
        return "☁️"  
    elif "rain" in description:  
        return "🌧️"  
    elif "clear" in description:  
        return "☀️"  
    elif "snow" in description:  
        return "❄️"  
    elif "storm" in description or "thunder" in description:  
        return "⚡️"  
    elif "mist" in description or "fog" in description:  
        return "🌫️"  
    else:  
        return "🔗"
```

Qodo Gen: Options | Test this function

```
def get_uv_index(lat, lon):  
    url = f"http://api.openweathermap.org/data/2.5/uvi?appid={API_KEY}&lat={lat}&lon={lon}"  
    response = requests.get(url)  
    data = response.json()  
  
    if response.status_code == 200:  
        return data.get("value", "N/A")  
    else:  
        return "N/A"
```

BREAKING NEWS

24/07

HURRICANE LUSI THIS WEEKEND

Qodo Gen: Options | Test this function

```
def plot_humidity(cities_data):  
    fig = go.Figure()  
  
    for data in cities_data:  
        if data:  
            fig.add_trace(go.Bar(  
                x=[data["city"]],  
                y=[data["humidity"]],  
                name=f"{data['city']} ({data['humidity']}%)"  
            ))
```

```
fig.update_layout(title="Humidity Comparison", xaxis_title="City", yaxis_title="Humidity (%)")  
fig.show()
```

Qodo Gen: Options | Test this function

```
def plot_forecast(city):  
    forecast_data = get_forecast(city)  
    if not forecast_data:  
        return  
  
    dates = [entry["datetime"] for entry in forecast_data]  
    temps = [entry["temperature"] for entry in forecast_data]  
    humidity = [entry["humidity"] for entry in forecast_data]  
  
    fig, axes = plt.subplots(2, 1, figsize=(12, 8))  
  
    sns.lineplot(x=dates, y=temps, marker='o', color='red', ax=axes[0])  
    axes[0].set_title(f"Temperature Forecast for {city}")  
    axes[0].set_ylabel("Temperature (°C)")  
    axes[0].tick_params(axis='x', rotation=45)  
  
    sns.lineplot(x=dates, y=humidity, marker='o', color='blue', ax=axes[1])  
    axes[1].set_title(f"Humidity Forecast for {city}")  
    axes[1].set_ylabel("Humidity (%)")  
    axes[1].tick_params(axis='x', rotation=45)  
  
    fig.tight_layout()  
    plt.show()
```

BREAKING NEWS

24/07

HURRICANE LUSI THIS WEEKEND

Qodo Gen: Options | Test this function

```
def get_forecast(city):
    url = f"http://api.openweathermap.org/data/2.5/forecast?q={city}&appid={API_KEY}&units=metric"
    response = requests.get(url)
    data = response.json()

    if response.status_code == 200:
        forecast_data = []
        for entry in data["list"]:
            dt_utc = datetime.strptime(entry["dt_txt"], "%Y-%m-%d %H:%M:%S").replace(tzinfo=timezone.utc)
            dt_ist = dt_utc.astimezone(IST).strftime("%Y-%m-%d %H:%M:%S")
            forecast_data.append({
                "datetime": dt_ist,
                "temperature": entry["main"]["temp"],
                "humidity": entry["main"]["humidity"]
            })
        return forecast_data
    else:
        print(f"Error fetching forecast for {city}: {data['message']}")
        return None
```

Qodo Gen: Options | Test this function

```
def schedule_weather_updates(cities):
    schedule.every(2).minutes.do(live_weather_update, cities=cities)

    while True:
        schedule.run_pending()
        time.sleep(1)

if __name__ == "__main__":
    cities = ["Hyderabad", "Nuzvid", "London", "Tokyo", "Delhi", "Vijaya"]
    city_for_forecast = input("Enter city name for forecast: ")
    if city_for_forecast not in cities:
        cities.append(city_for_forecast)

    live_weather_update(cities)
    plot_forecast(city_for_forecast)
    schedule_weather_updates(cities)
```


Tree ▾

object ▶ main ▶

▼ object {14}

▼ coord {2}

lon : 7.367

lat : 45.133

▼ weather [1]

▼ 0 {4}

id : 501

main : Rain

description : moderate rain

icon : 10d

base : stations

▶ main {8}

visibility : 10000

▶ wind {3}

▶ rain {1}

▶ clouds {1}

dt : 1726660758

▶ sys {5}

timezone : 7200

id : 3165523

name : Province of Turin

cod : 200

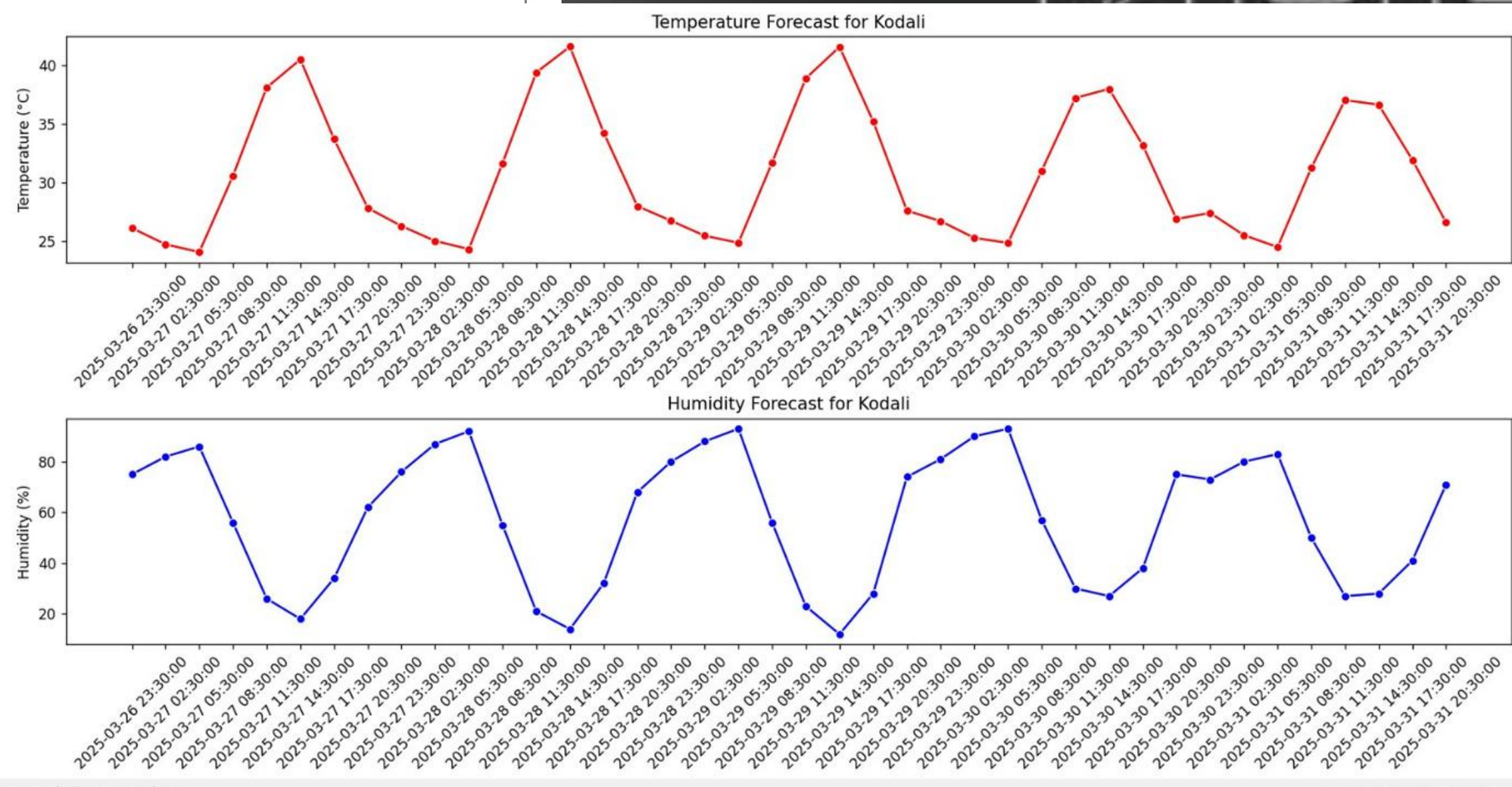
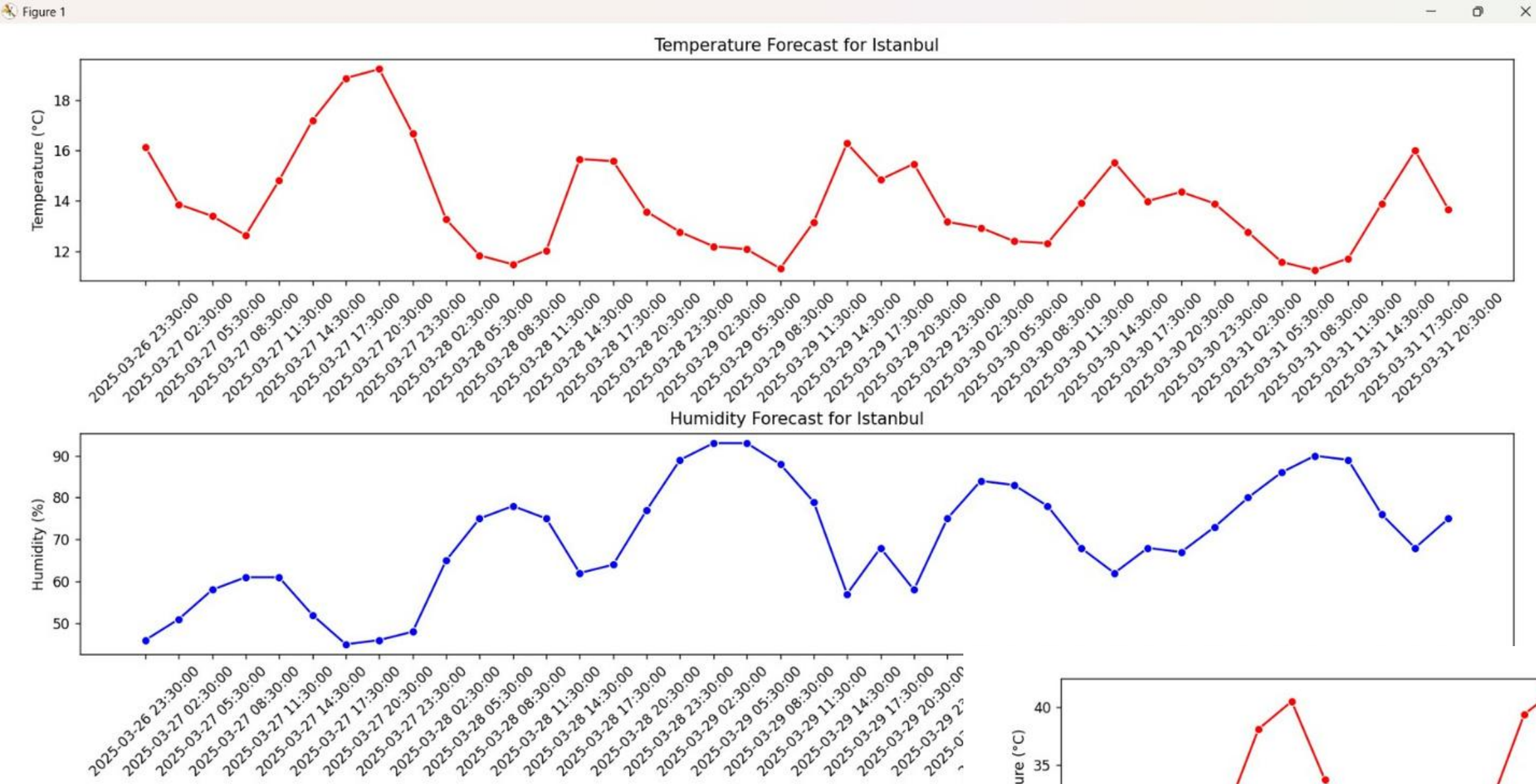
JSON format API response example

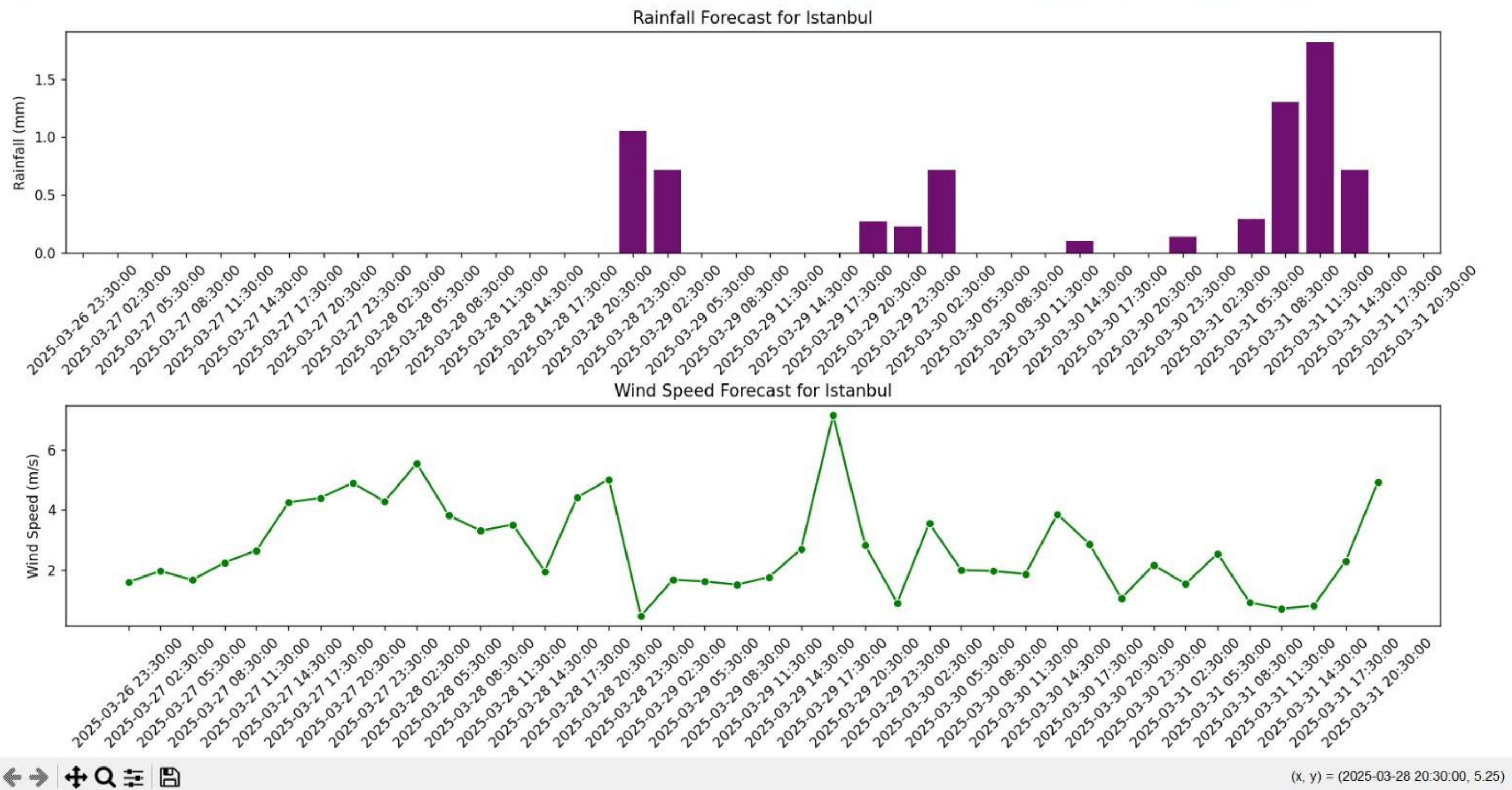
```
{
  "coord": {
    "lon": 7.367,
    "lat": 45.133
  },
  "weather": [
    {
      "id": 501,
      "main": "Rain",
      "description": "moderate rain",
      "icon": "10d"
    }
  ],
  "base": "stations",
  "main": {
    "temp": 284.2,
    "feels_like": 282.93,
    "temp_min": 283.06,
    "temp_max": 286.82,
    "pressure": 1021,
    "humidity": 60,
    "sea_level": 1021,
    "grnd_level": 910
  },
  "visibility": 10000,
  "wind": {
    "speed": 4.09,
    "deg": 121,
    "gust": 3.47
  }
}
```

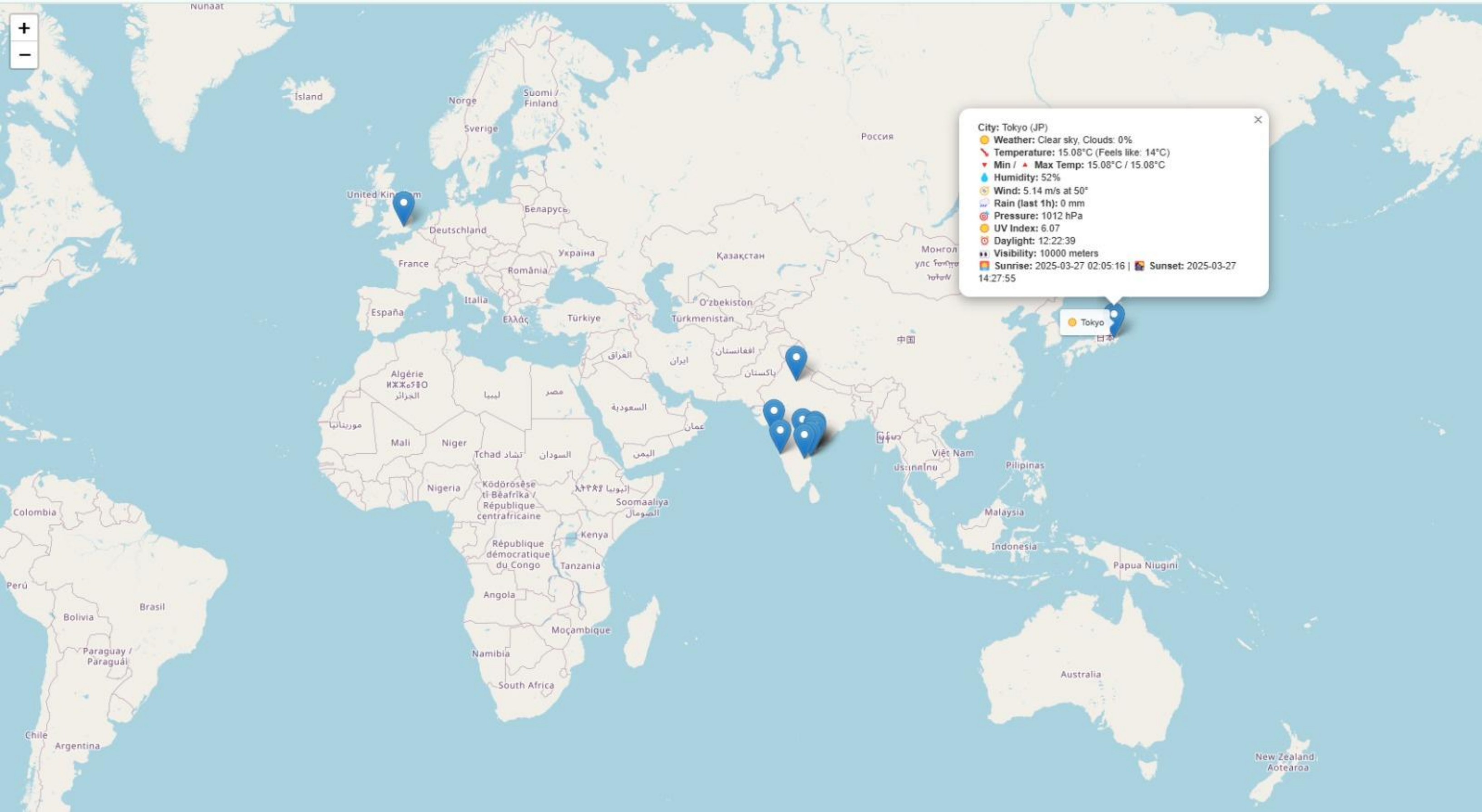
BREAKING NEWS

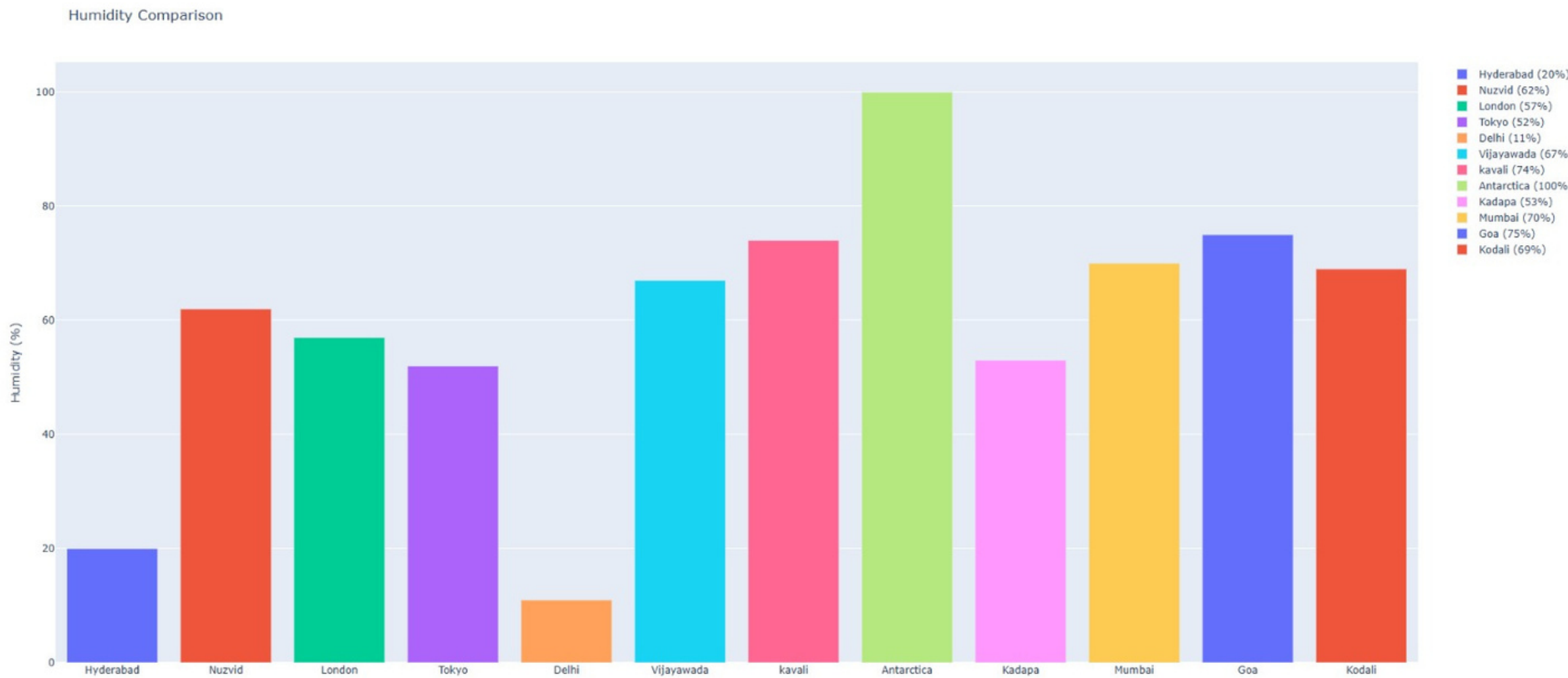
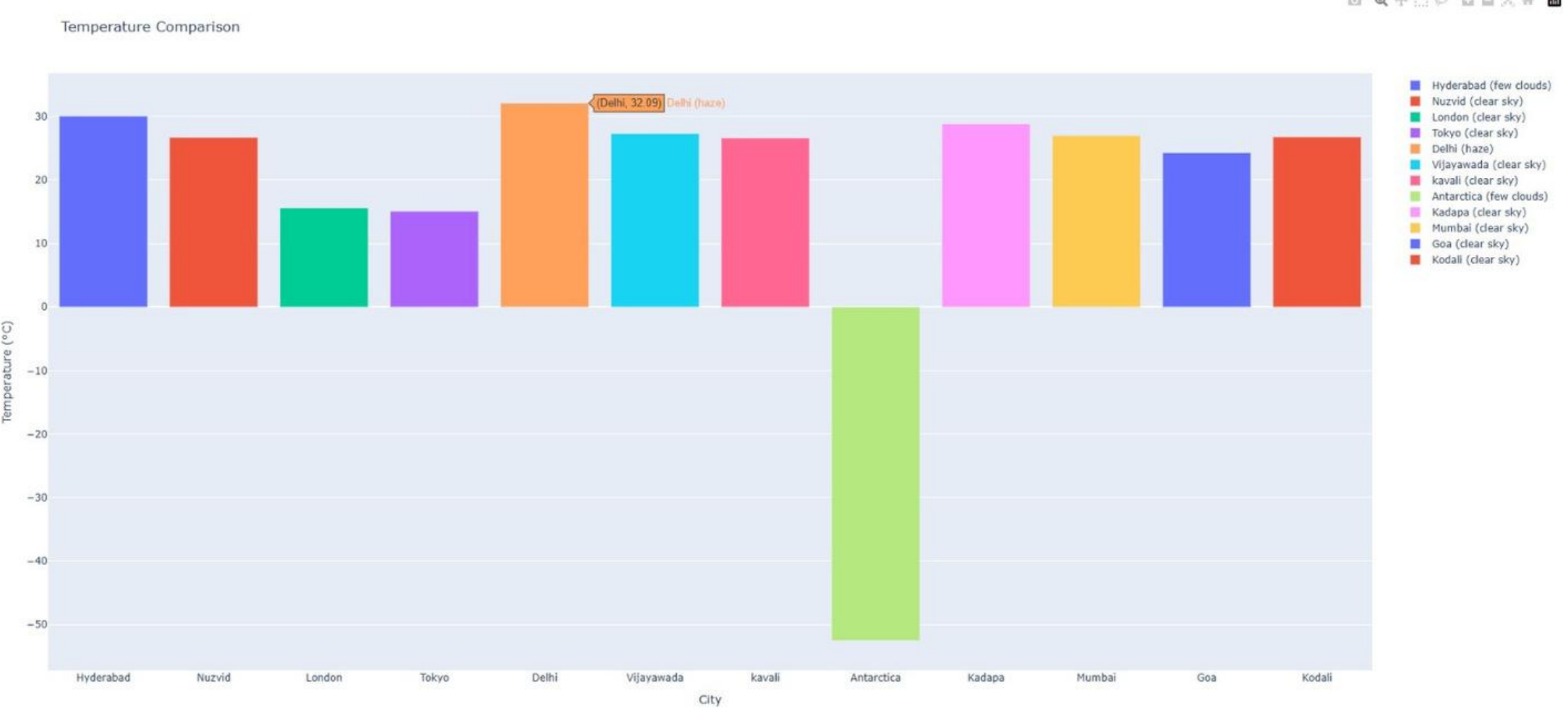
24/07

HURRICANE LUSI THIS WEEKEND









OUTCOMES

Accurate Weather Retrieval:

- Successfully fetches real-time weather data for multiple cities.
- Provides additional insights like wind chill, UV index, and daylight duration.

Timezone Handling

- Converts UTC timestamps to Indian Standard Time (IST) for better readability.

Improved User Experience

- Uses emojis for quick weather interpretation.
- Displays temperature status (Cold, Pleasant, Hot).

Data Logging & History

- Weather data is saved in a text file (weather_log.txt), allowing tracking over time.

Interactive Data Visualisation

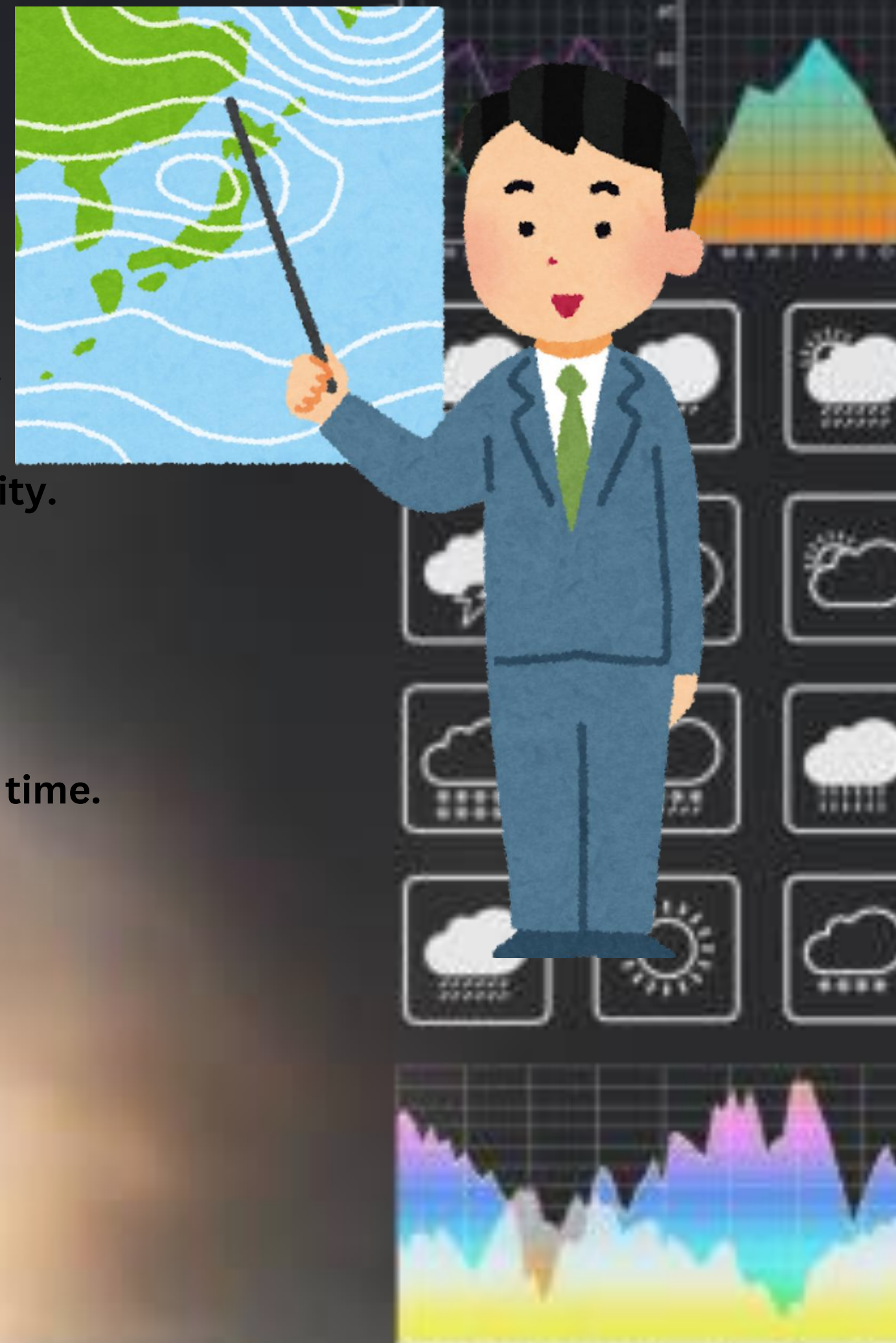
- Matplotlib & Seaborn create trend graphs for temperature changes.
- Folium supports map-based visualization.
- Bokeh & Plotly provide interactive charts.

Automated Updates

- Uses schedule to fetch live weather data every 10 minutes.

Scalability

- Supports multiple cities and allows adding new cities dynamically



CHALLENGES FACED

API Limitations

- Free-tier APIs have rate limits, restricting the number of requests per minute.
- Some features (e.g., UV index) may require a paid plan.

Handling API Failures

- Some cities may not return data due to incorrect spelling or missing records.
- Implemented error handling (`response.status_code`) to manage API failures.

Time Zone Adjustments

- Converting UTC to IST was necessary for user-friendliness.
- Handled time formatting to ensure proper visualization.

Data Visualisation Complexity

- Forecast graphs required proper handling of timestamps.
- Seaborn and Matplotlib plots needed rotation of x-axis labels for clarity.

Real-time Updates

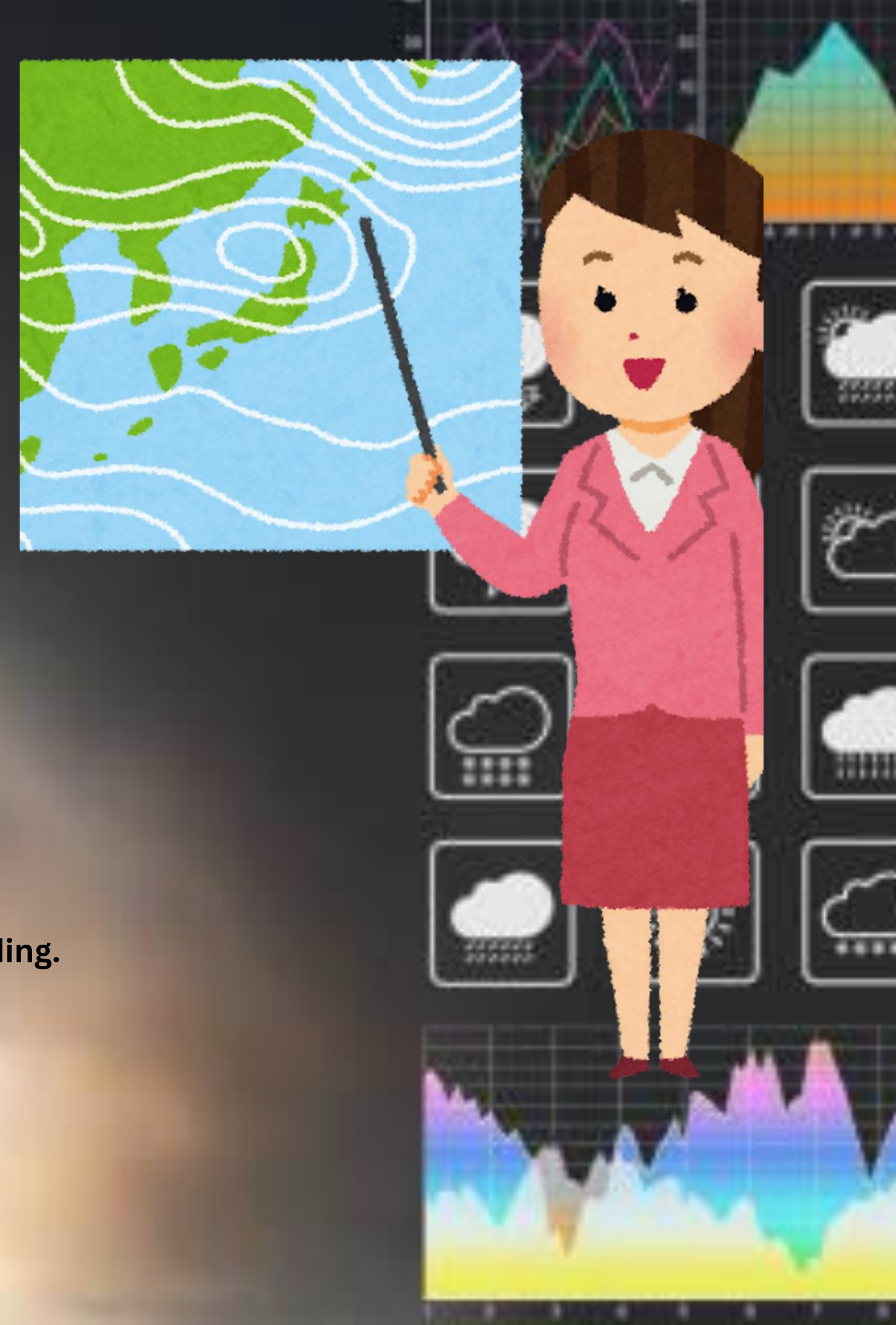
- Continuous API requests can slow down execution.
- Used `schedule` and `time.sleep(10)` to ensure periodic updates without overloading.

Wind Chill Calculation Constraints

- The formula only applies under specific conditions (temperature $\leq 10^{\circ}\text{C}$).
- Returns "N/A" when conditions are not met.

User Input Handling

- Users may enter an invalid city name, causing API errors.
- Implemented a city validation step before making API calls.



Conclusions

Future Enhancements:

- 🚀 Integrate machine learning models for weather trend predictions
- 🚀 Improve mobile compatibility by developing a web-based dashboard
- 🚀 Include push notifications for severe weather alerts

Final Thoughts

This weather forecasting project successfully integrates real-time weather data retrieval, data visualization, and automation to provide an insightful and user-friendly experience. The script fetches weather data for multiple cities, logs historical records, and presents the data in interactive charts, maps, and text reports.

BREAKING NEWS

24/07

HURRICANE LUSI THIS WEEKEND

HURRICANE CENTRAL RADAR

RAIN

HEAVY RAIN

11:50AM ET

THANK YOU

TRACKING SANDY

EMERG. DECLARATIONS SUPPLEMENT
STATE, LOCAL RESOURCES

KEY BISCAYNE

11:52

69°

The Weather Channel